

PVB

Programare Vizuală prin Blocuri

Aplicație nativă de programare vizuală, cu generare automată de cod sursă real (C++ și Python) și legătură biunivocă bloc ↔ linie de cod

Autor: Costan Matei - Ștefan

Liceu: Colegiul Național "I. L. Caragiale" - București

Profesor coordonator: Florea Andrei

GitHub: github.com/mcostn/pvb

Cuprins

Alegerea tematicii	3
I.1 Analiza pieței	3
Elemente distinctive față de soluțiile existente	4
I.2 Planificarea dezvoltării	6
Implementarea aplicației	7
II.1 Proiectarea arhitecturală	7
Definirea blocurilor	8
Convertirea blocurilor în cod sursă	11
Paradigme de programare folosite	12
II.2 Tehnologiile folosite	12
II.3 Stabilitatea aplicației	13
II.4 Securitatea aplicației	14
II.5 Testarea produsului	15
II.6 Maturitatea aplicației	16
II.7 Folosirea sistemului de versionare	18
II.8 Ghid de instalare, compilare și configurare	19
Instalare din GitHub Releases (recomandat)	19
Linux	19
Windows	19
Compilarea aplicației din codul sursă	19
Cerințe	19
Clonarea repository-ului	20
Linux	20
Windows	20
Generarea unei versiuni distribuite	20
Interfața	21
III.1 Interfața grafică	21
III.2 Experiența utilizatorului	23
Lucrul în echipă	27
IV.1 Distribuția rolurilor	27
IV.2 Modul de lucru	28
Resurse Externe	29
Concluzii și opinia autorului	30

Alegerea tematicii

I.1 Analiza pieței

Programarea vizuală prin blocuri este una dintre cele mai eficiente metode de a introduce noțiuni de algoritmică unor persoane fără experiență prealabilă în scriere de cod, tocmai pentru că elimină bariera sintaxei. Cele mai cunoscute soluții din acest domeniu sunt Scratch (MIT Media Lab) și librăria Blockly (care este folosită de Scratch, dar și de mulți alții, cum ar fi PBInfo și Microsoft MakeCode). Sunt instrumente educaționale valoroase și larg folosite, însă au o limitare comună fundamentală: blocurile vizuale nu au nicio legătură directă, vizibilă, cu un limbaj de programare textual real. Elevul care a învățat să gândească “în blocuri” trebuie, la un moment dat, să facă un salt brusc către sintaxa unui limbaj precum C++ sau Python, fără ca platforma să îl fi pregătit pentru acest pas.

PVB (Programare Vizuală prin Blocuri) pornește exact de la această observație: este gândit ca o punte între gândirea vizuală și codul real, nu ca un înlocuitor al acestuia. Funcția centrală a aplicației, Generate Code, transformă în timp real orice ansamblu de blocuri de pe canvas într-un fișier sursă valid în C++ sau Python, la alegerea utilizatorului. Fiecare bloc este legat de linia sa de cod corespunzătoare printr-o hartă sursă (source map) proprie, astfel încât utilizatorul poate observa direct, bloc cu bloc, cum se traduce o structură de control sau o expresie într-o construcție de limbaj.

Elemente distinctive față de soluțiile existente

Caracteristică	Blockly (PBInfo)	Scratch	MakeCode	Stencyl	PVB
Programare prin blocuri	✓	✓	✓	✓	✓
Compilare și rulare integrată	✓	✓	✓	✓	✓
Blocuri personalizate	✗	✓	✓	✓	✓
Generare de cod real, compilabil	✗	✗	JavaScript și Python	Haxe (un limbaj care nu prea mai este folosit)	C++ și Python
Legătură bloc ↔ linie de cod	✗	✗	✗	✗	✓
Salvare/încărcare proiect local	✗	✗ (necesită cont)	✗ (necesită cont)	✓	✓
Funcționare complet offline	✗	✗	✗	✓	✓
Editor randat cu accelerație hardware	✗	✗	✗	✓	
Licență	Proprietară	GPLv2 + BSD	Proprietară	Proprietară	GPLv3
Cost	Gratuit	Gratuit	Gratuit	licență comercială pentru export avansat	Gratuit

Avantajul-cheie al PVB

Funcția Generate Code convertește instantaneu blocurile vizuale în cod C++ sau Python valid, cu o legătură biunivocă (source map) între fiecare bloc și linia sa de cod. Profesorii pot construi o asocierie concretă între schema vizuală și sintaxa reală a unui limbaj, iar orice începător poate observa efectul fiecărei acțiuni fără a mai citi un manual de sintaxă înainte de a scrie prima linie de cod.

Spre deosebire de majoritatea mediilor de programare vizuală, precum pbinfo Blockly, Scratch sau Microsoft MakeCode, care sunt dezvoltate în principal ca aplicații web, PVB este o aplicație nativă compilată direct pentru sistemul de operare al utilizatorului. Interfața este randată hardware prin OpenGL, oferind o experiență fluidă chiar și pentru proiecte complexe și eliminând dependența de un browser web. Dintre aplicațiile analizate, Stencyl este, de asemenea, o soluție nativă, însă aceasta utilizează ecosistemul Haxe, ceea ce presupune instalarea și configurarea unor dependențe suplimentare pentru compilare și generează cod într-un limbaj mai puțin întâlnit în programele educaționale și în competițiile de algoritmică. În plus, interfața și fluxul de lucru ale aplicației au rămas în mare parte neschimbate de mai mulți ani, fiind orientate către compatibilitate și stabilitate, mai degrabă decât către o experiență modernă de utilizare. Prin comparație, PVB este conceput în jurul limbajelor C++ și Python, folosite pe scară largă în mediul educațional și în industrie, oferă o interfață modernă și funcționează complet offline, fără a necesita servicii externe sau configurări suplimentare. Distribuirea proiectului sub licența GNU GPLv3 și disponibilitatea gratuită a codului sursă permit atât utilizarea, cât și studierea sau extinderea aplicației în scop educațional.

Publicul țintă al PVB este reprezentat, în primul rând, de profesorii de informatică care predau bazele algoritmicii și doresc un instrument vizual care să rămână permanent conectat la sintaxa unui limbaj de programare real. Aplicația se adresează și elevilor sau studenților aflați la începutul studiului programării, oferindu-le posibilitatea de a construi algoritmi prin blocuri și de a observa simultan codul C++ sau Python generat. Astfel, utilizatorii pot înțelege treptat corespondența dintre reprezentarea vizuală și implementarea textuală, reducând dificultatea tranziției către programarea clasică și eliminând teama de erorile de sintaxă specifice primelor contacte cu un limbaj de programare.

I.2 Planificarea dezvoltării

Dezvoltarea PVB a parcurs mai multe etape iterative, fiecare validând sau infirmând o ipoteză tehnică înainte de a trece la implementarea finală:

- **Prototip 1: Parser în C:** un parser care transforma un AST (Abstract Syntax Tree) minimal direct în cod C++. Acest prototip a validat fezabilitatea ideii centrale, dar limbajul C s-a dovedit prea rigid pentru structurile de date necesare (definiții de blocuri, scheme de argumente, arbori de expresii).
- **Prototip 2: Renderer 2D propriu în OpenGL:** s-a explorat construirea unui motor grafic 2D de la zero pentru randarea blocurilor. Deși funcțional, efortul de implementare (gestionarea manuală a input-ului, a textului, a layout-ului) era disproporționat față de valoarea adăugată, comparativ cu folosirea unei biblioteci UI mature.
- **Decizia finală:** C++20 pentru flexibilitatea și performanța pe care le oferă față de C, combinat cu Dear ImGui ca strat de randare a interfeței. Logica de interacțiune (drag & drop, conectarea blocurilor, evaluarea expresiilor) a fost în continuare implementată integral de la zero; ImGui a fost folosit exclusiv pentru desenarea elementelor pe ecran, nu pentru logica lor.

Planul de dezvoltare a fost organizat incremental, pornind de la un nucleu minim funcțional (un singur bloc Main, generare de cod C++ pentru instrucțiuni simple) și extins treptat, funcționalitate cu funcționalitate, fiecare urmărită printr-un commit dedicat în istoricul Git (vezi secțiunea II.7). Ordinea aproximativă a fost: blocuri de bază și generare C++ → structuri de control (if/else, while, for) → variabile și expresii → compilare și rulare integrată → blocuri custom (subprograme) → suport pentru Python ca al doilea limbaj țintă → salvare/încărcare proiect → rafinări de UX (scalare canvas, comentarii, redenumire, ștergere).

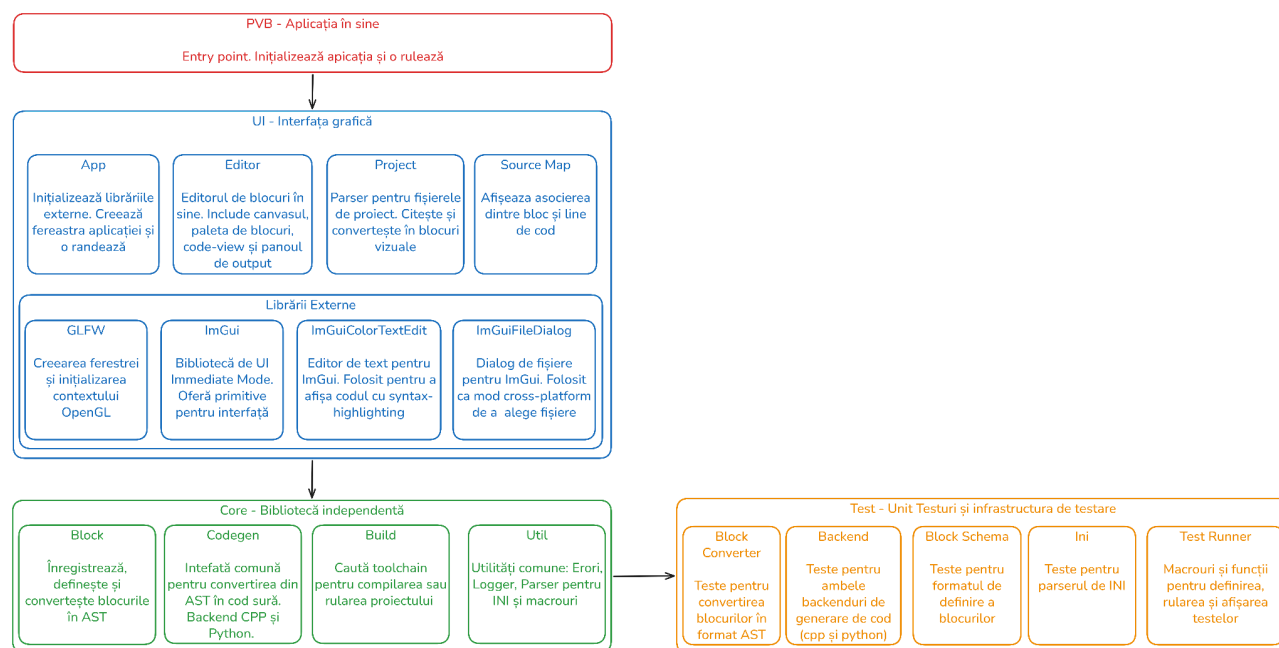
Această abordare incrementală a permis testarea continuă a fiecărei componente noi imediat după implementare (vezi II.5 Testarea produsului), reducând riscul ca o funcționalitate nouă să strice comportamentul celor deja existente.

Implementarea aplicației

II.1 Proiectarea arhitecturală

PVB este structurat pe trei biblioteci separate, legate printr-o singură direcție de dependență, ceea ce ține logica de generare a codului complet independentă de interfața grafică:

- **core**: definirea și înregistrarea blocurilor (block/), transformarea lor într-un AST (**block/converter**), cele două backend-uri de generare de cod, C++ și Python (codegen/), și integrarea cu toolchain-ul de compilare/rulare (build/). Nu depinde de nicio bibliotecă grafică și este singura bibliotecă folosită și de suita de teste automate.
- **ui**: tot ce ține de interfața grafică: canvas-ul, blocurile desenate, paleta laterală, editorul de cod cu evidențiere sintactică, dialogurile de salvare/încărcare a proiectului și harta sursă vizuală. Depinde de core pentru a converti blocurile în cod, dar core nu are nicio cunoștință despre ui.
- **pvb (executabil)**: punctul de intrare care inițializează fereastra OpenGL/GLFW și pornește bucla principală a interfeței.



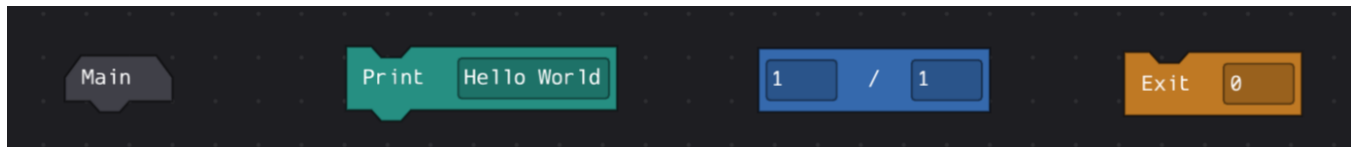
Arhitectura pe straturi a PVB: interfață (ui), nucleu logic (core, independent de UI) și suita de testare.

Varianta mai mare [aici](#).

Această separare aduce un avantaj concret, validat deja în practică: adăugarea limbajului Python ca al doilea backend de generare de cod (`codegen/py_backend`) nu a necesitat nicio modificare a interfeței grafice sau a logicii blocurilor, ci doar a fost suficient un nou modul care implementează aceeași interfață abstractă de backend (`codegen/backend.hpp`), alături de cel existent pentru C++ (`codegen/cpp_backend`).

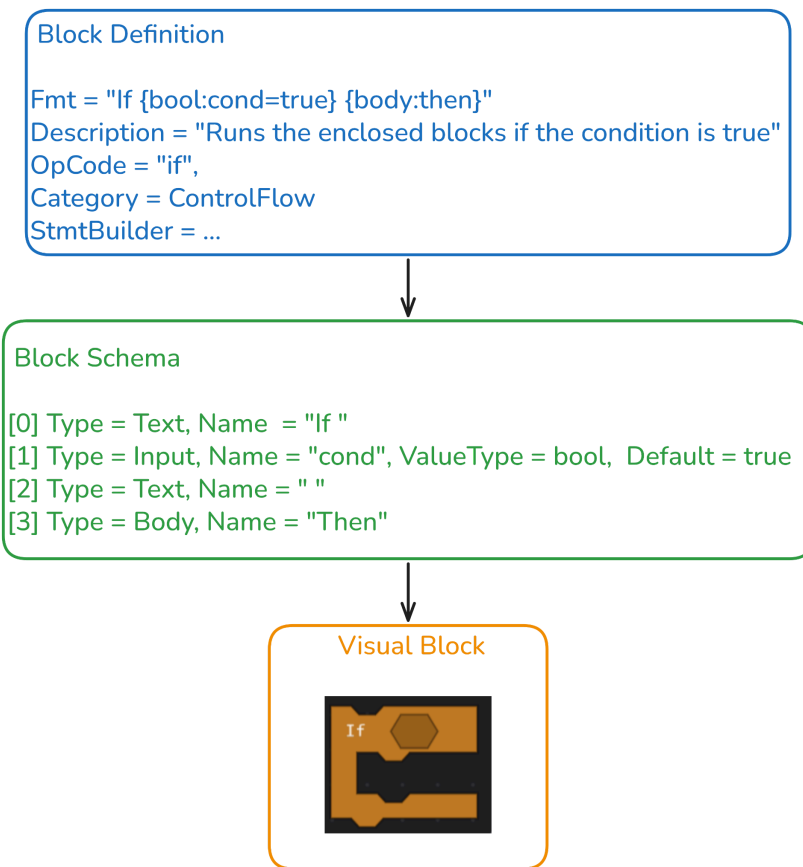
Definirea blocurilor

Fiecare tip de bloc este descris declarativ printr-o structură `BlockDefinition`, formată dintr-un identificator unic (`OpCode`), un șablon textual (`Fmt`) care descrie simultan textul afișat pe fața blocului și sloturile sale de intrare, o categorie (`BlockCategory`), o descriere opțională, o formă vizuală (`BlockShape`: Chain, Hat, Cap sau Reporter) și, cel mai important, un constructor de AST: un `StmtBuilder` pentru blocurile-instrucțiune sau un `ExprBuilder` pentru blocurile-expresie, transmise ca `std::function`. Toate definițiile sunt colectate într-un `BlockRegistry`, care expune metoda `RegisterBlock` pentru înregistrare și păstrează atât lista blocurilor disponibile (`Definitions`), cât și maparea dintre opcode-uri și constructorii lor de AST, folosită ulterior de `BlockConverter`.



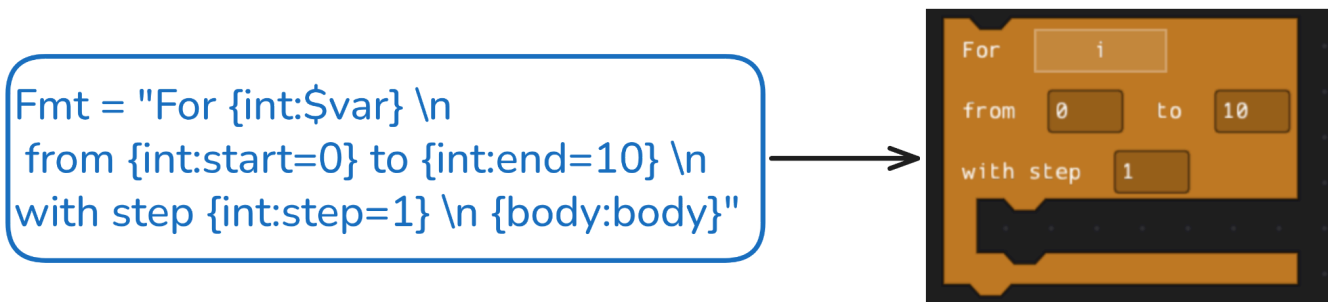
Toate formele posibile de blocuri. De la stânga la dreapta: main are format Hat, print are forma Chain, împărțirea are forma Reporter, iar exit are forma Cap.

Câmpul `Fmt` este o mică sintaxă proprie, interpretată la înregistrarea blocului, care transformă un șir precum „`If {bool:cond=true} {body:then}`” într-o listă structurată de sloturi (`BlockSchema`). Textul din afara acoladelor devine un slot `Text`, afișat ca atare pe fața blocului; `{tip:nume=valoare_implicită}` devine un slot `Input`, cu un tip din setul `int`, `float`, `bool`, `string`, `number` sau `any` și, opțional, o valoare implicită; `{tip:$nume}` (cu prefixul `$`) devine un slot `Var`, care leagă blocul de o variabilă existentă în proiect, nu de o valoare literală; iar `{body:nume}` devine un slot `Body`, marcând un container de instrucțiuni, folosit de blocurile de control (`if`, `if-else`, `while`, `for`).



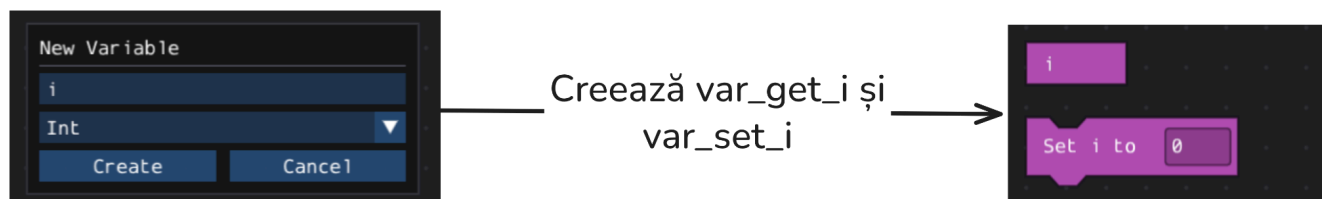
Definiția blocului if: Transformarea BlockDefintion $\rightarrow$ BlockSchema $\rightarrow$ VisuaBlock și randarea lui

Un rând nou în Fmt introduce un slot LineBreak, ceea ce permite blocuri pe mai multe linii, precum for. Același parser stabilește și numele argumentelor (de exemplu „cond” sau „then”) pe care StmtBuilder-ul sau ExprBuilder-ul blocului le va citi din BlockInstance::Args la conversia în AST.



Blocul for care conține sloturi de LineBreak, pentru economisirea spațiului orizontal

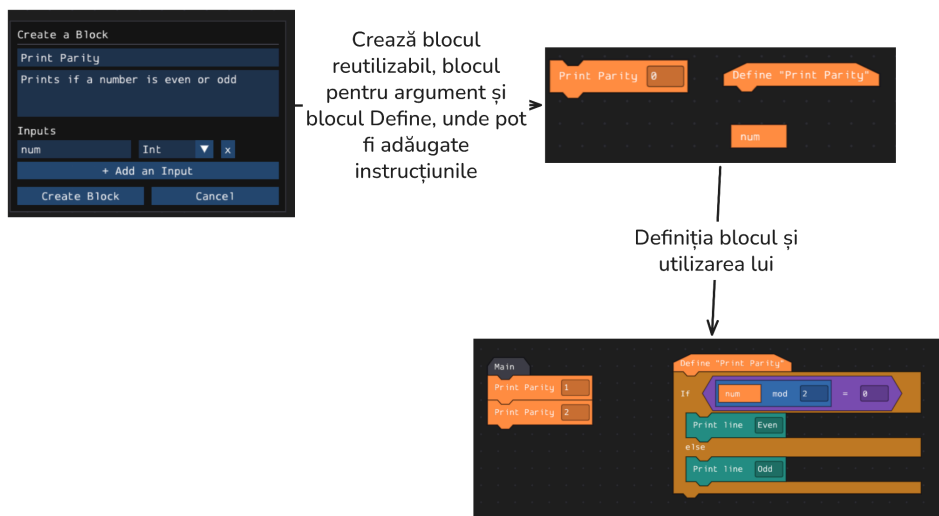
OpCode este identificatorul unic după care **RegisterBlock** respinge înregistrările duplicate (**Error::BlockAlreadyExists**) și după care **BlockConverter** caută constructorul potrivit de AST în momentul convertirii unui bloc de pe canvas. Definițiile de bloc nu sunt întotdeauna statice: blocurile pentru variabile sunt generate dinamic (construind la runtime atât opcode-urile (**var_get_<nume>** și **var_set_<nume>**, prin funcțiile), cât și șirurile **Fmt** și lambda-urile **ExprBuilder/StmtBuilder** corespunzătoare, în funcție de numele și tipul variabilei create de utilizator în interfață.



Crearea variabilei i

Pe lângă blocurile predefinite, PVB permite utilizatorului să definească propriile blocuri prin intermediul funcționalității **Make a Block**. Aceasta oferă posibilitatea de a grupa o secvență de instrucțiuni într-o abstracție reutilizabilă, similară unei funcții sau proceduri din limbajele de programare clasice.

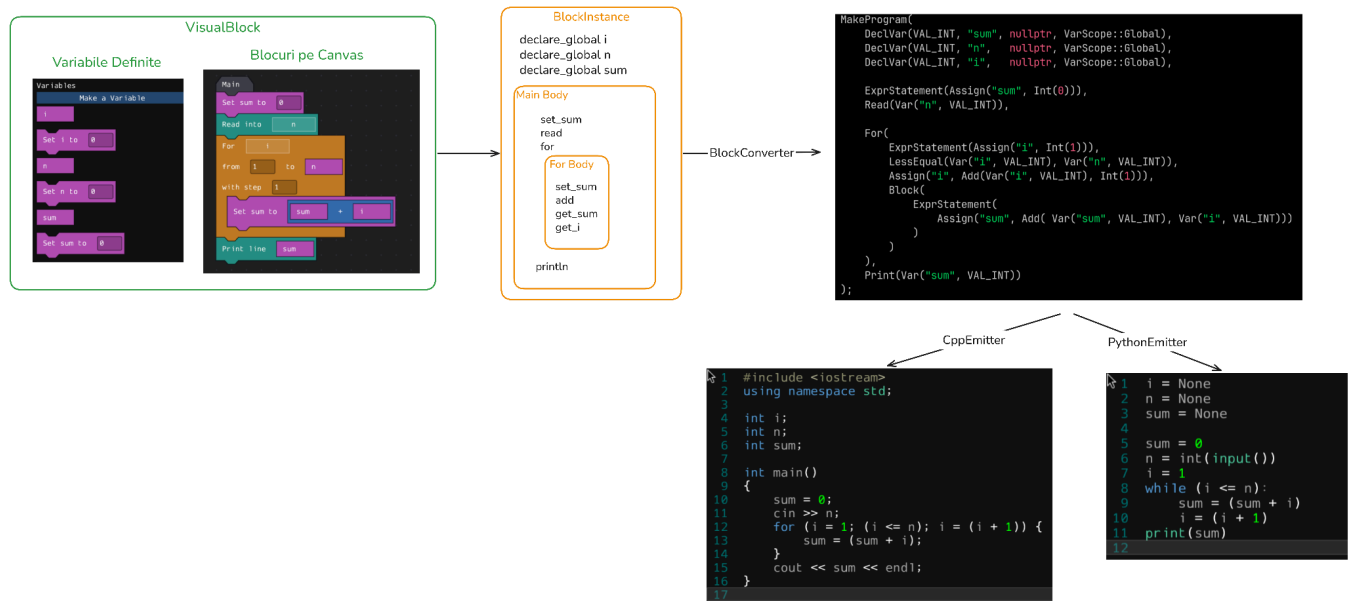
În timpul definirii unui bloc nou, utilizatorul stabilește numele acestuia, lista parametrilor. După confirmare, editorul generează automat definiția blocului și îl adaugă în biblioteca de blocuri. Secvența de blocuri care definește blocul personalizat va fi adăugată de către runtime în **StmtBuilder** înainte de compilarea sau rularea proiectului.



Crearea unui bloc personalizat care afișează dacă un număr este par sau nu. Variantă mai mare [aici](#).

Din punct de vedere conceptual, blocurile create de utilizator sunt tratate în același mod ca blocurile native ale limbajului. Ele dispun de o definiție proprie, pot primi parametri, pot conține alte blocuri și participă în mod identic la procesul de generare a codului sursă. Această uniformitate simplifică arhitectura aplicației și permite extinderea limbajului fără modificarea nucleului editorului.

Convertirea blocurilor în cod sursă



Fluxul de conversie: blocurile de pe canvas devin un AST, care este parcurs de un backend de cod (C++ sau Python) pentru a produce sursa finală. Variantă mai mare [aici](#).

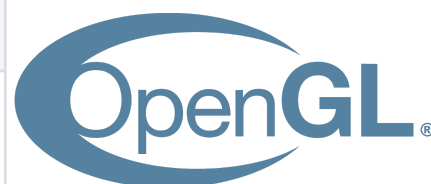
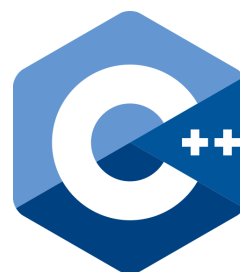
Conversia propriu-zisă are loc în trei pași. Blocurile de pe canvas sunt reprezentate cu structura `VisualBlock`, care este convertită în `BlockInstance`. După, `BlockConverter` parcurge lista de `BlockInstance` și le transformă într-un arbore de sintaxă abstractă (declarații și expresii), independent de limbajul țintă. Apoi, fiecare backend de cod (`CppEmitter` sau `PythonEmitter`) parcurge acest arbore și produce codul sursă corespunzător, împreună cu o hartă sursă ce asociază fiecare nod din arbore cu un bloc și cu linia de cod generată.

Paradigme de programare folosite

- Programare orientată pe obiecte pentru modelarea blocurilor, a categoriilor și a componentelor de interfață (Canvas, Block, Palette, Editor).
- Programare funcțională, prin constructori de blocuri (`StmtBuilder` / `ExprBuilder`) transmiși ca `std::function` la înregistrarea fiecărui tip de bloc - fiecare bloc “știe” cum să se transforme în AST fără ca restul sistemului să cunoască detaliile specifice.
- Propagare explicită a erorilor inspirată de Zig, printr-un `enum Error` verificat cu atributul `[[nodiscard]]` și un macro `TRY(expr)` care întrerupe execuția și propagă eroarea în sus pe stivă imediat ce apare, fără excepții C++.

II.2 Tehnologiile folosite

Tehnologie	Rol în proiect
C++20	Limbaj principal; control complet asupra memoriei, funcționalitate modernă (structured bindings, concepts, ranges, format).
Dear ImGui	Bibliotecă UI immediate-mode; randează interfața, dar logica de drag & drop, conectare și expresii e scrisă integral în proiect.
GLFW + OpenGL	Crearea ferestrei native și accelerarea hardware a randării canvas-ului.
ImGuiColorTextEdit	Editor de text cu colorare sintactică C++/Python pentru panoul Code View, cu suport pentru evidențierea liniei curente (necesar hărții sursă).
ImGuiFileDialog	Dialoguri de tip “Save As” / “Open” pentru salvarea și încărcarea proiectelor <code>.pvp</code> .
CMake	Sistem de build cross-platform (Linux și Windows) din aceeași bază de cod, cu suport pentru ccache și pentru unity build pe bibliotecile core și ui.



Justificarea alegerii tehnologiilor: C++ a fost preferat față de C (folosit în primul prototip) pentru că oferă abstractizări de nivel înalt (clase, templates, `std::variant`, `std::format`) fără a renunța la controlul asupra memoriei, esențial pentru o aplicație care manipulează în timp real liste de blocuri și arbori de sintaxă. Dear ImGui a fost ales în locul unui framework retained-mode (precum Qt sau Gtk) pentru că modelul immediate-mode se potrivește natural cu un canvas dinamic, unde blocurile se creează, se șterg și se reconectează constant - nu există stare de UI persistentă de sincronizat manual. În schimb, tocmai pentru că Dear ImGui nu oferă primitive pentru “blocuri conectabile”, întreaga logică de snap, drag și validare a conexiunilor a trebuit implementată de la zero, ceea ce a oferit control deplin asupra comportamentului, dar a mărit semnificativ complexitatea codului din biblioteca ui. CMake a fost ales pentru portabilitatea sa reală între Linux și Windows, testată constant pe parcursul dezvoltării.

II.3 Stabilitatea aplicației

PVB este o aplicație nativă, compilată, fără dependențe de runtime grele (nu folosește un browser, o mașină virtuală sau un motor JavaScript). Alegerea Dear ImGui (o bibliotecă minimalistă, fără dependențe externe grele) combinată cu randarea accelerată prin OpenGL, menține consumul de resurse redus chiar și pe hardware modest, tipic pentru laboratoarele școlare.

Gestionarea memoriei este predominant automată folosind funcționalități moderne de C++. Blocurile de pe canvas și nodurile arborelui de sintaxă sunt administrate prin containere STL (`std::vector`, `std::deque`, `std::unordered_map`, `std::unique_ptr` acolo unde e nevoie de proprietate exclusivă), evitând alocările repetate în bucla de randare.

Biblioteca core (unde se află logica de generare de cod și cea de build) este complet decuplată de interfața grafică, ceea ce înseamnă că o eroare de randare sau de interacțiune UI nu poate corupe starea logică a proiectului, și invers.

Compilarea și rularea codului generat se fac în procese externe separate (`build/process`, `build/runner`), astfel încât un program C++ sau Python scris greșit de utilizator nu poate bloca sau doborî aplicația PVB: output-ul și codul de ieșire al procesului sunt capturate și afișate în panoul Run Log.

Pe parcursul dezvoltării, aplicația a fost rulată și verificată periodic pe Linux (X11, Wayland) și Windows, fără blocaje sau creșteri anormale de consum de memorie observate în utilizare susținută (sesiuni cu zeci de blocuri pe canvas).

II.4 Securitatea aplicației

PVB este o aplicație de desktop, offline, fără nicio formă de comunicare în rețea, ceea ce elimină de la bază o întreagă categorie de vulnerabilități (injecții prin rețea, exfiltrare de date, atacuri de tip man-in-the-middle). Riscurile relevante pentru acest tip de aplicație țin, în schimb, de validarea datelor introduse de utilizator și de gestionarea proceselor externe pe care aplicația le pornește pentru a compila și a rula codul generat.

Propagarea erorilor este explicită și obligatorie: toate funcțiile care pot eșua (înregistrarea unui bloc, parsarea unui fișier `.pvb`, pornirea unui proces de compilare) returnează o enumerație `Error` marcată cu `[[nodiscard]]`, astfel încât compilatorul semnalează orice loc din cod care ignoră o eroare posibilă. Macro-urile `TRY`, `FAIL_COND_V` și `FAIL_COND_V_MSG` propagă eroarea imediat și afișează un mesaj descriptiv prin loggerul global.

```
enum class [[nodiscard]] Error
{
    Ok = 0,
    Failed,
    Unreachable,
    ...
};
```

```
std::ifstream File(Path);

FAIL_COND_V_MSG(!File.is_open(),
    Error::FileNotFound,
    "Failed to open '{}'",
    Path);
```

Definiția prescurtată a enumerației `Error` și un exemplu de folosire. `FAIL_COND_V_MSG` va returna `Error::FileNotFound` dacă fișierul nu a putut fi deschis și va afișa mesajul folosind `std::format`

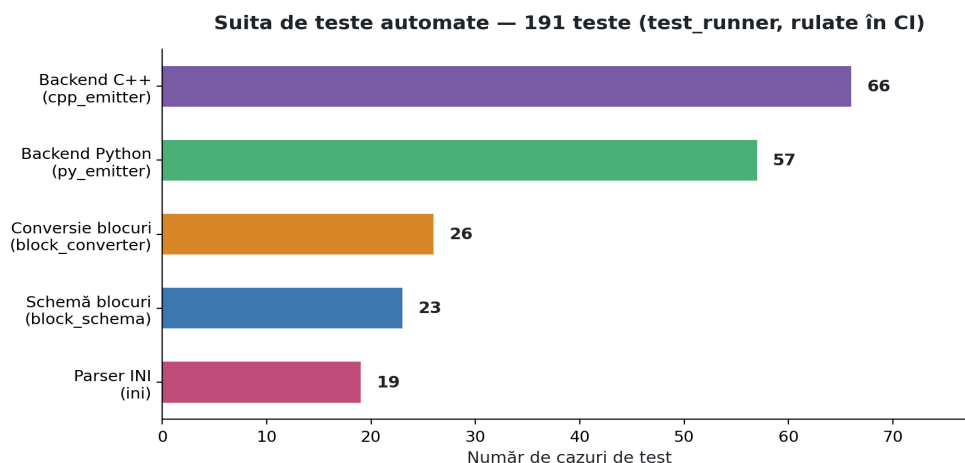
Validarea intrărilor: numele de variabile și de blocuri custom sunt verificate înainte de înregistrare (nu pot fi goale, nu pot coincide cu un opcode deja existent), iar valorile din câmpurile editabile ale blocurilor sunt validate în funcție de tipul asociat (int, float, bool, string) înainte de a fi trimise generatorului de cod, prevenind emiterea de cod C++/Python invalid.

Fișierele de proiect (`.pvb`, format INI propriu) sunt parsate printr-un parser dedicat cu coduri de eroare specifice pentru secțiuni sau chei invalide, duplicate sau lipsă, iar încărcarea unui proiect cu o versiune de format necunoscută (câmpul `pvb_version`) este respinsă în mod explicit, în loc să fie interpretată eronat.

Codul sursă generat este scris pe disc și compilat/rulat prin apeluri explicite de proces (nu prin eval sau interpretare in-process), iar rezultatul (fie erori de compilare de la g++/clang++/MSVC, fie output-ul programului la rulare) este afișat integral utilizatorului în panourile Build Log și Run Log, fără a ascunde vreo informație de diagnostic.

II.5 Testarea produsului

Proiectul include o suită de teste automate proprii, construită de la zero (`tests/test_runner.hpp`), fără dependență de un framework extern de testare. Aceasta oferă un mecanism minimal, dar suficient, de înregistrare a testelor, comparare a rezultatelor și raportare a eșecurilor cu fișier și linie exactă. Suita rulează integral pe biblioteca core, complet independent de interfața grafică, ceea ce permite rularea ei automată pe orice mașină, inclusiv într-un mediu fără server grafic (headless), cum este cel folosit de integrarea continuă.



Distribuția celor 191 de teste automate pe cele cinci module testate.

Teste pe backend-ul C++ și pe backend-ul Python: fiecare test construiește un program mic direct sub formă de AST (fără a trece prin blocuri) și verifică textul sursă generat caracter cu caracter, acoperind include-uri automate, structuri de control, apeluri de funcții matematice, blocuri custom și edge case-uri (de exemplu, expresii ca instrucțiuni în interiorul unui for).

Teste de conversie a blocurilor: verifică transformarea corectă a unei instanțe de bloc (`BlockInstance`), cu argumentele și conexiunile ei, în nodul de AST corespunzător.

Teste de schemă a blocurilor: validează parsarea formatului de definire a unui bloc (câmpul `Fmt`, ex. “`Set {num:value=0}`”) în sloturi de tip `Text`, `Input`, `Var` și `Body`.

Teste ale parser-ului INI: acoperă citirea și scrierea formatului folosit pentru fișierele de proiect `.pvb`, inclusiv cazuri invalide care trebuie respinse.

Integrare continuă: un workflow GitHub Actions (`.github/workflows/tests.yml`) compilează proiectul cu opțiunea `BUILD_GUI=OFF` și rulează automat suita de teste la fiecare push și la fiecare pull request, împiedicând introducerea de regresii în cele două backend-uri de generare de cod sau în parsere.

Testare manuală și de portabilitate: pe lângă testele automate, fiecare tip de bloc a fost testat individual și în combinații complexe direct din interfață, iar aplicația a fost compilată și rulată atât pe Linux, cât și pe Windows, folosind același sistem de build CMake, fără modificări de cod sursă între platforme.

II.6 Maturitatea aplicației

PVB se află într-un stadiu funcțional avansat: toate elementele planificate inițial au fost implementate și testate. Tabelul de mai jos arată evoluția față de planul inițial de dezvoltare.

Funcționalitate planificată inițial	Stadiu actual
Blocuri de bază: Events, Console, Control Flow, Math, Logic, Variables	✓ Implementat (extins și cu o categorie String)
Generare cod C++ cu source map și compilare integrată	✓ Implementat
Structuri repetitive: blocuri While și For	✓ Implementat (For cu pas configurabil)
Blocuri custom (subprograme) definite de utilizator	✓ Implementat (definire, apel, redenumire, ștergere)
Salvare și încărcare proiect	✓ Implementat (format <code>.pvb</code> propriu, dialog nativ)
Suport pentru Python ca al doilea limbaj țintă	✓ Implementat

Odată acest plan inițial finalizat, au apărut și funcționalități suplimentare, neplanificate inițial, dar utile din feedback-ul de utilizare: variabile globale, comentarii atașabile direct pe canvas, un sistem de scalare (zoom) al canvas-ului, o paletă de blocuri redimensionabilă și un meniu contextual (click-dreapta) pentru duplicarea și ștergerea blocurilor.

Baza de utilizatori este, în acest moment, limitată la un cerc restrâns de beta-testeri (un elev de gimnaziu, un elev de liceu și un profesor de informatică), însă feedback-ul primit confirmă ipoteza centrală a proiectului: asocierea directă dintre un bloc vizual și linia de cod pe care o generează reduce vizibil curba de învățare a sintaxei.

“Am înțeles mult mai ușor cum funcționează un if în C++ când am văzut că blocul If-then generează exact acel cod. Era clar vizual ce face fiecare piesă.”

- Elev, clasa a VI-a, beta-tester

“Asocierea dintre blocuri vizuale și secvențe C++ oferă o metodă ușoară de înțelegere a algoritmilor elementari pentru începătorii în acest domeniu.”

- Elev, clasa a IX-a, beta-tester

“Ar fi util să am asta la clasă când introduc variabilele și structurile de control. Elevii pot experimenta vizual și vedea imediat codul echivalent.”

- Profesor de informatică, beta-tester

Roadmap de dezvoltare

Deși funcționalitățile planificate inițial au fost implementate, arhitectura modulară a aplicației permite extinderea acesteia fără modificări majore ale nucleului de generare de cod. În continuare sunt avute în vedere următoarele direcții de dezvoltare:

Funcționalitate	Stadiu
Structuri de date complexe (vectori unidimensionali și multidimensionali, operații de acces și modificare)	Planificat
Tablouri dinamice și containere STL (std::vector, list, map) pentru C++	Planificat
Funcții cu valori returnate	Planificat
Website dedicat proiectului , care să includă documentație completă, tutoriale și exemple de utilizare pentru profesori și elevi	Planificat
Import și export Scratch pentru interoperabilitate	În analiză
Suport pentru limbaje suplimentare de programare	În analiză

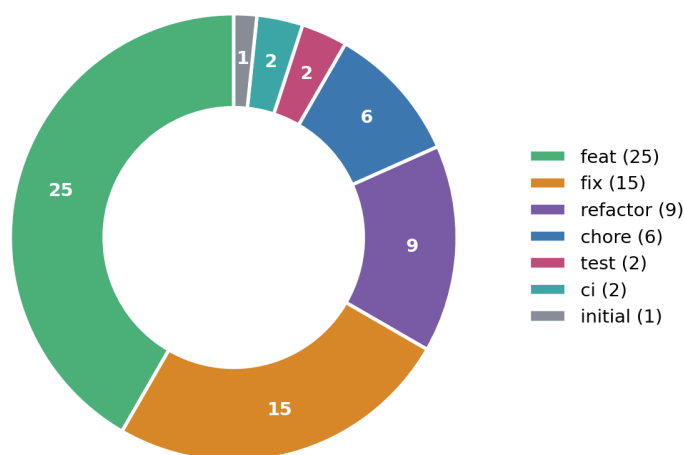
II.7 Folosirea sistemului de versionare

Proiectul folosește Git, cu repository public pe GitHub (github.com/mcostn/pvb).

Istoricul de commit-uri urmează convenția Conventional Commits, cu prefixe clare pentru natura fiecărei schimbări:

- **feat**: o funcționalitate nouă (ex. feat: source map, feat: string blocks, feat: rename variables and custom blocks)
- **fix**: corectarea unui defect existent (ex. fix: use canvas zoom level when dragging a block from the palette)
- **refactor**: restructurare fără schimbarea comportamentului (ex. refactor: decouple block from canvas)
- **chore** / **test** / **ci**: configurare, adăugare de teste sau de workflow-uri de integrare continuă

Istoricul commit-urilor Git (60 total)
după tip (Conventional Commits)



Distribuția celor 60 de commit-uri din istoricul Git, pe tipuri de schimbare.

Această disciplină de denumire face istoricul ușor de parcurs și permite identificarea rapidă a commit-ului care a introdus o anumită funcționalitate sau regresie. Fiecare stare intermediară a proiectului rămâne accesibilă prin istoricul Git, permițând revenirea la orice versiune anterioară dacă este necesar. În plus, workflow-ul de integrare continuă descris la II.5 rulează automat la fiecare push, oferind un semnal imediat dacă o schimbare a introdus o regresie.

II.8 Ghid de instalare, compilare și configurare

Aplicația poate fi utilizată fie prin compilarea codului sursă, fie prin descărcarea unei versiuni precompilate din [GitHub Releases](#).

Instalare din GitHub Releases (recomandat)

Linux

1. Accesați secțiunea [GitHub Releases](#) a proiectului.
2. Descărcați arhiva pentru Linux (de exemplu, `pvb-linux-x86_64.tar.gz`).
3. Dezarhivați pachetul: `tar -xzf pvb-linux-x86_64.tar.gz`
4. Intrați în directorul rezultat și executați aplicația: `./pvb`

Windows

1. Accesați secțiunea [GitHub Releases](#) a proiectului.
2. Descărcați arhiva destinată platformei Windows (de exemplu, `pvb-windows-x86_64.zip`).
3. Dezarhivați conținutul într-un director la alegere.
4. Executați fișierul `pvb.exe` prin dublu-click sau din Command Prompt

Compilarea aplicației din codul sursă

Cerințe

- **Git** pentru clonarea repository-ului;
- **CMake 3.16** sau o versiune mai nouă;
- Un compilator compatibil cu standardul **C++20**:
 - **Linux:** GCC sau Clang;
 - **Windows:** Microsoft Visual Studio 2022 (MSVC) sau MinGW-w64.



Clonarea repository-ului

Repository-ul utilizează submodule Git, astfel încât acesta trebuie clonat recursiv:

```
git clone --recursive https://github.com/mcostn/pvb.git  
cd pvb
```

Dacă repository-ul a fost clonat fără submodule, acestea pot fi descărcate ulterior cu:

```
git submodule update --init --recursive
```

Linux

1. Inițializați directorul de compilare: `./init.sh`
Pentru o compilare în modul **Release**: `./init.sh --release`
2. Compilați aplicația: `cmake --build build --parallel`
3. Executabilul va fi generat în directorul `build`.

Windows

1. Inițializați directorul de compilare: `init.bat`
Pentru o compilare în modul **Release**: `init.bat --release`
2. Compilați aplicația: `cmake --build build --parallel`.
Dacă compilați cu MSBuild (MSVC) și vreți **Release**, trebuie să folosiți:
`cmake --build build --config Release --parallel`
3. Executabilul rezultat va fi disponibil în directorul `build`.

Generarea unei versiuni distribuite

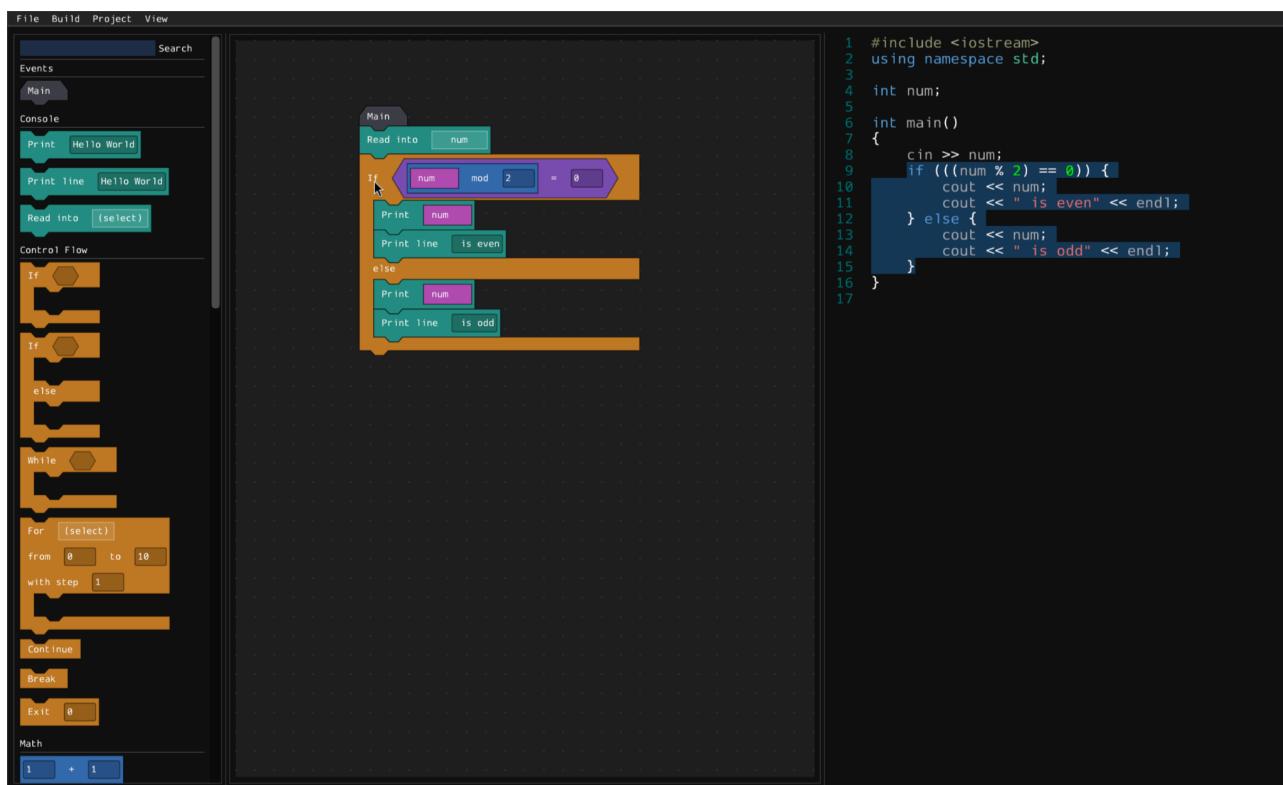
Proiectul include scripturile `release.sh` (Linux) și `release.bat` (Windows), care automatizează procesul de creare a unui pachet distribubil pentru publicarea unei noi versiuni în **GitHub Releases**. Acestea configurează proiectul în modul **Release**, compilează aplicația, copiază executabilul și exemplele într-un director dedicat și generează arhiva distribubilă, pregătită pentru încărcare în GitHub Releases.

Interfața

III.1 Interfața grafică

Interfața PVB a fost concepută pentru claritate și eficiență, punând utilizatorul în contact direct cu blocurile, fără elemente decorative inutile. Logica și randarea blocurilor (poziționare, conectare (snap), drag & drop, evaluarea sloturilor de expresie) au fost implementate integral de la zero; nu există nicio bibliotecă third-party pentru aceste comportamente, Dear ImGui fiind folosit exclusiv ca strat de randare de bază (butoane, câmpuri de text, ferestre).

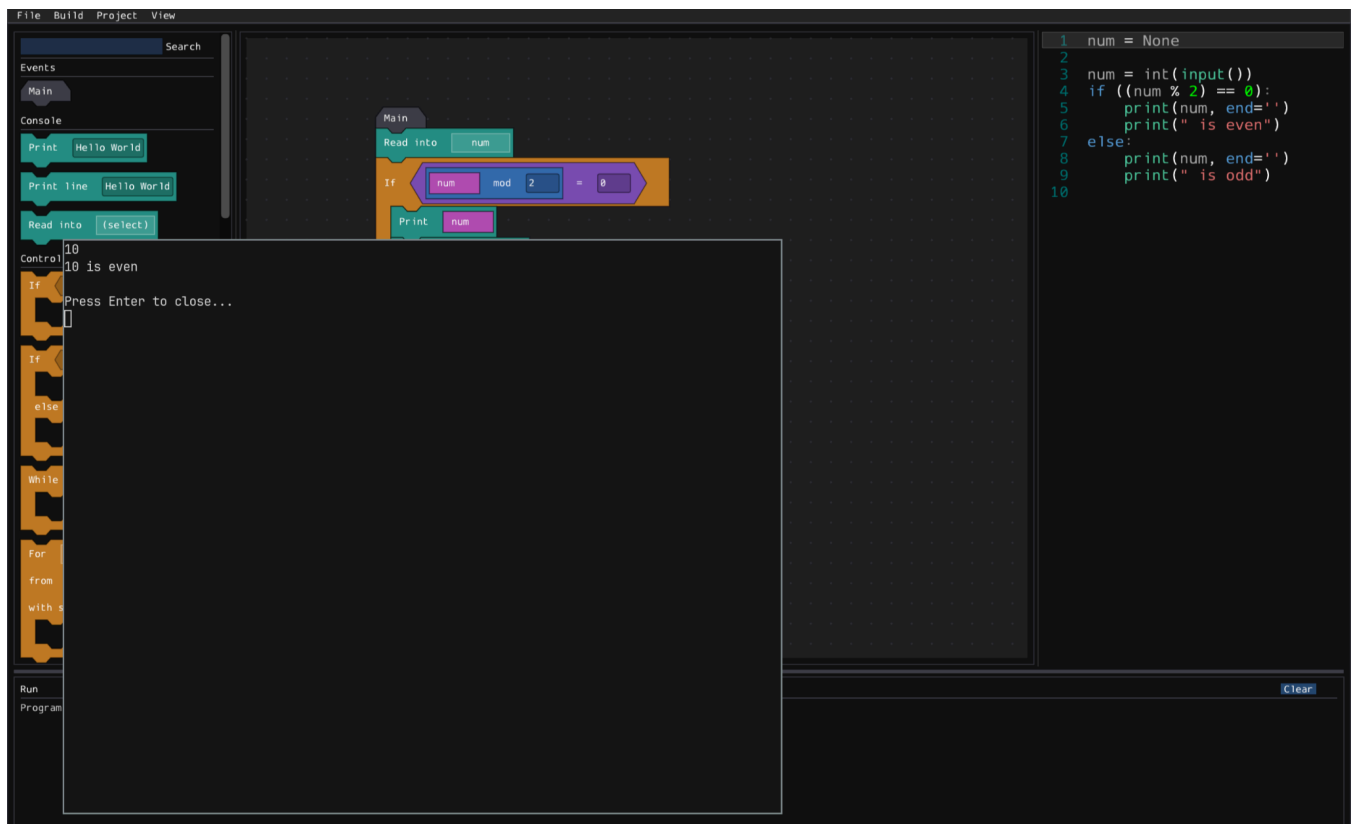
Aplicația se organizează în jurul a trei zone principale, vizibile simultan: paleta de blocuri din stânga (organizată pe categorii cromatice), canvas-ul central, unde se construiește programul, și, opțional, panoul de cod generat din dreapta, cu evidențiere sintactică și scroll sincronizat cu blocul selectat prin harta sursă.



Editorul cu panoul de cod deschis, și codul C++ generat. Deoarece utilizatorul a plasat cursorul deasupra construcției if, CodeView subliniază tot codul generat aferent blocului (folosind harta sursă)

Fiecare categorie de blocuri păstrează o culoare distinctivă pe tot parcursul aplicației: verde pentru evenimente (Main), albastru pentru consolă (Write, Read into, End Line), portocaliu pentru control flow (If, While, For), mov pentru operații matematice, cyan pentru logică și roz pentru variabile, oferind recunoaștere vizuală instantanee, indiferent de complexitatea programului construit.

Interfața respectă principii clasice de UI: consistență vizuală (fiecare categorie de blocuri își păstrează culoarea peste tot), feedback imediat (blocurile afișează un contur de previzualizare înainte de a se conecta), și densitate informațională controlată: panoul de cod, panoul de output și sidebar-ul pot fi ascunse independent din meniul View, pentru a lăsa loc canvas-ului atunci când nu sunt necesare.

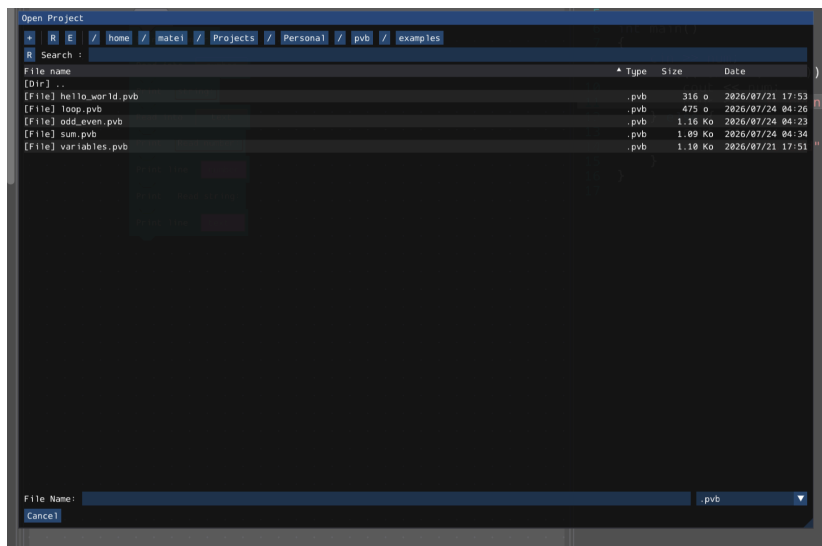
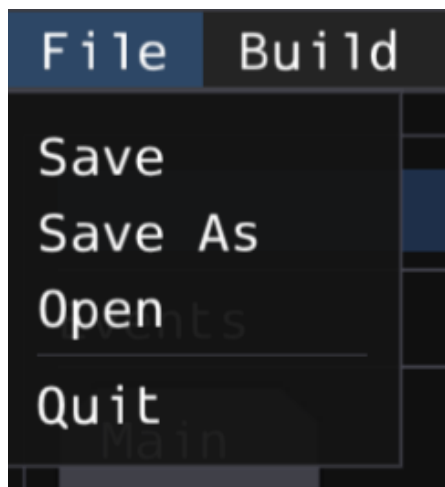


Editorul cu toate panourile deschise în timp de programul rulează într-un terminal separat

III.2 Experiența utilizatorului

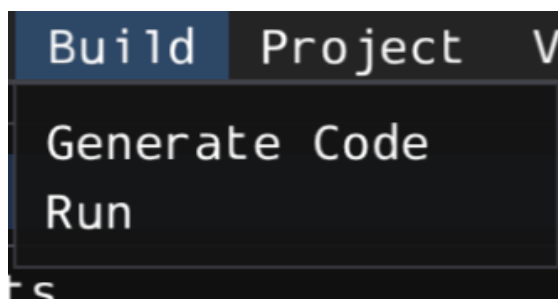
Fiecare componentă a interfeței a fost gândită pentru a reduce fricțiunea și a menține fluxul de lucru cât mai apropiat de gândirea naturală “construiesc, apoi văd rezultatul”:

Meniul File oferă Save, Save As, Open și Quit pentru gestionarea proiectelor .pvb salvate local, prin dialoguri native de fișiere.



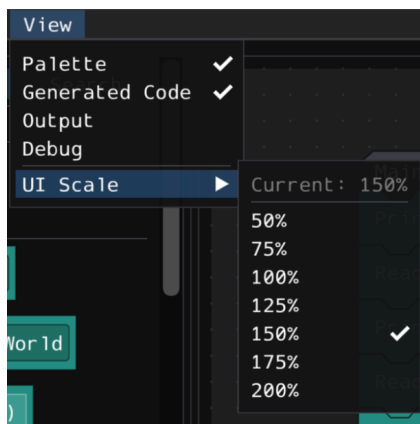
Meniul File și dialogul de fișiere pentru a deschide un proiect.

Meniul Build grupează Generate Code (transpilare în limbajul curent) și Run (compilarea dacă e nevoie și execuția programului rezultat).



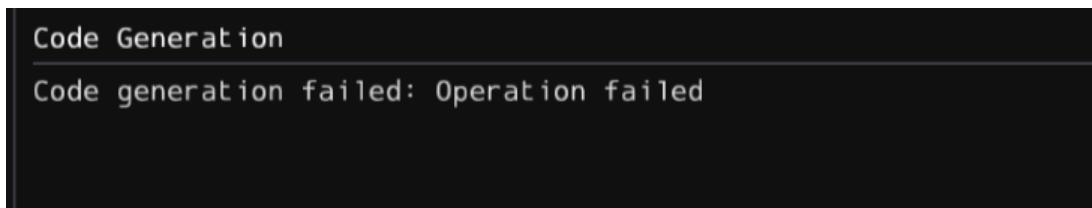
Meniul Build cu opțiunile sale.

Meniul View comută vizibilitatea sidebar-ului, a panoului de cod, a panoului de output, ferestre de debug (pentru dezvoltatori sau în cazul în care cineva întâmpină un bug), și opțiunea de mărire sau micșorare a interfaței lăsând utilizatorul să adapteze layout-ul în funcție de sarcină.

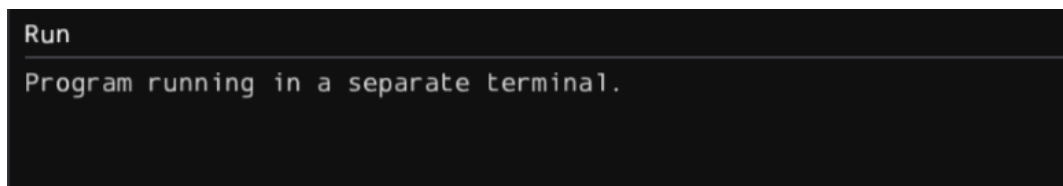


Meniul View cu opțiunile de UI Scale.

Panoul de Output, afișează starea de rulare sau compilare.



Panoul de Output când generarea codului eșuează.

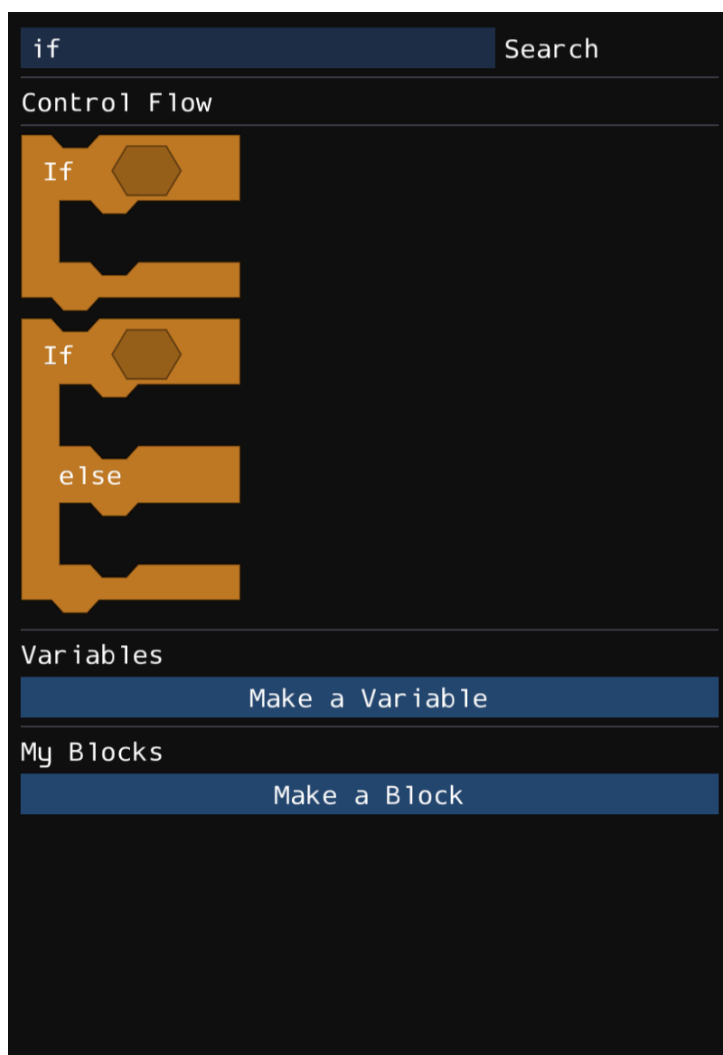


Panoul de Output în timp ce un program rulează.

Paleta de blocuri reprezintă principalul mod de interacțiune cu aplicația și conține toate blocurile disponibile pentru construirea algoritmilor. Acestea sunt organizate pe categorii funcționale (Control Flow, Variables, Math, Logic etc.), fiecare fiind identificată printr-o culoare specifică, ceea ce facilitează navigarea și identificarea rapidă a blocurilor necesare. Acestea pot fi și filtrate atât după denumirea afișată a blocului (**Fmt**), cât și după identificatorul său intern (**OpCode**)



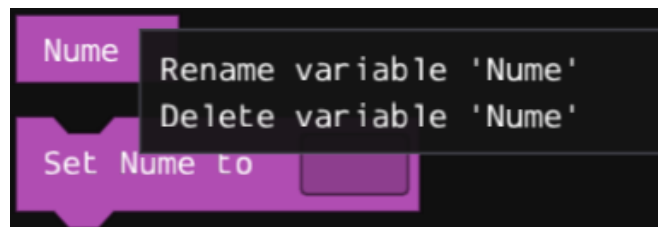
Paleta cu mai multe categorii vizibile



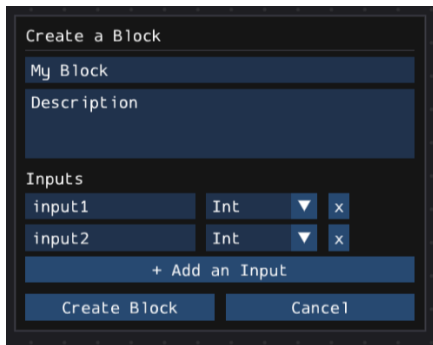
Paleta unde utilizatorul caută construcția “if”



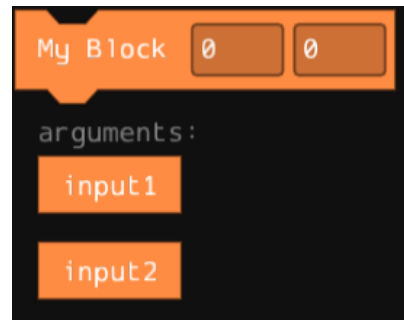
Dialogul de creare a unei variabile cu selectarea tipului



Blocurile rezultate din dialogul anterior. Utilizatorul poate redenumi sau șterge variabile

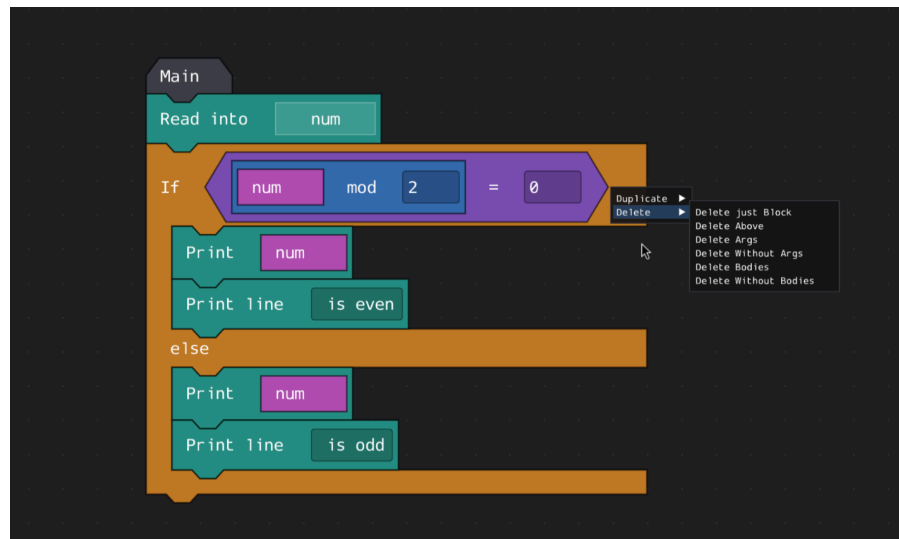


Dialogul de creare a unui bloc personalizat



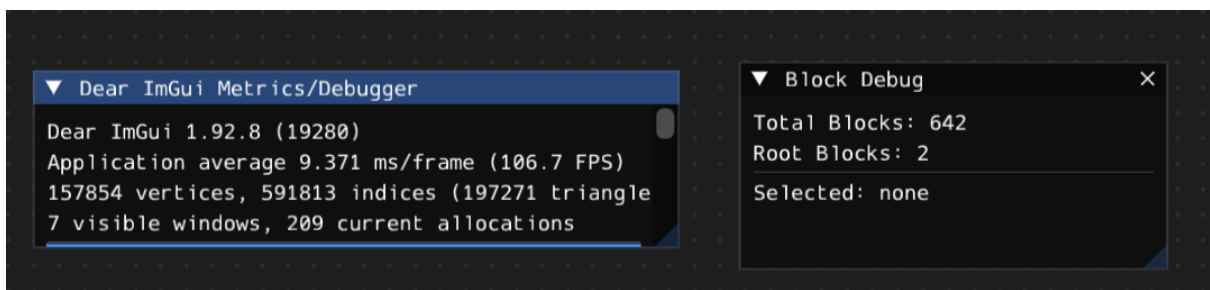
Blocul creat, sub care se află parametrii lui

Canvas-ul suportă panning prin click (middle mouse button) & drag pe fundal, scalare (prin scroll), și un meniu contextual (click-dreapta) pe orice bloc, cu opțiunile Delete și Duplicate.



Editorul doar cu canvasul vizibil și meniul contextual cu acțiuni avansate pentru blocul selectat (if)

Un aspect important pentru experiența utilizatorului este viteza de reacție: fiind o aplicație nativă, randată direct prin OpenGL, interfața rămâne fluidă chiar și atunci când canvas-ul conține zeci de blocuri interconectate, fără întârzierile de randare specifice interfețelor bazate pe DOM/browser.



Ferestrele de Debug vizibile cu statisticele pentru 642 de blocuri pe Canvas

Lucrul în echipă

IV.1 Distribuția rolurilor

PVB este dezvoltat de un singur autor, Costan Matei-Ștefan, care a acoperit integral toate rolurile implicate în realizarea proiectului: proiectarea arhitecturii pe straturi, implementarea logicii de blocuri și a canvas-ului, scrierea celor două backend-uri de generare de cod (C++ și Python), construirea suitei de teste automate, integrarea continuă și redactarea documentației. Istoricul Git confirmă acest lucru direct: toate cele commit-uri din repository aparțin aceluiași autor.

Absența unei echipe a impus, în compensare, o disciplină individuală mai strictă: fiecare schimbare a fost făcută printr-un commit dedicat, cu mesaj descriptiv, iar funcționalitățile mai mari (de exemplu, suportul pentru Python sau blocurile custom) au fost implementate incremental, în mai multe commit-uri succesive, fiecare menținând proiectul într-o stare compilabilă și testată.



Statistic repo din 21 Iunie până pe 19 Iulie

IV.2 Modul de lucru

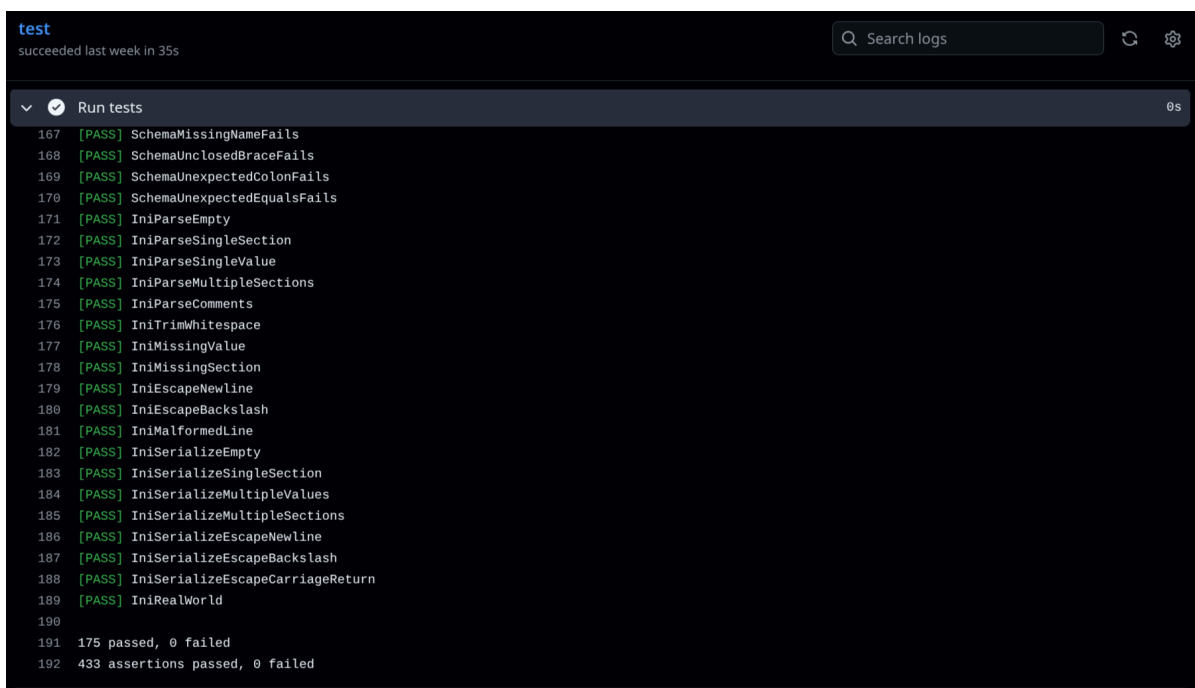
Chiar și în lipsa unei echipe propriu-zise, dezvoltarea a fost organizată cu instrumente și practici standard de inginerie software, pentru a păstra un flux de lucru trasabil și repetabil:

Sistem de versionare Git, cu istoric public pe GitHub: fiecare stare intermediară a proiectului este recuperabilă, iar mesajele de commit urmează convenția Conventional Commits, descrisă în detaliu la II.7.

Integrare continuă (GitHub Actions): la fiecare push sau pull request, se compilează automat proiectul (fără interfața grafică, pentru a rula în medii fără server grafic) și se execută integral suita de teste, oferind un semnal imediat de regresie, exact rolul pe care l-ar juca într-o echipă un proces de code review sau de bug tracking automatizat.

Structurare modulară a codului sursă (secțiunea VI.1): separarea clară pe module (block/, codegen/, build/, ui/, util/) face codul ușor de navigat și de extins independent, o precondiție necesară dacă proiectul ar fi preluat, pe viitor, de o echipă mai mare.

Documentație bilingvă a proiectului (README.md în engleză și docs/README-RO.md în română), care conține instrucțiuni complete de clonare, configurare și build, utile oricărui colaborator viitor care s-ar alătura proiectului.



The screenshot shows a GitHub Actions workflow named 'test' that succeeded last week in 35s. The workflow is running on a 'Run tests' step, which has a duration of 0s. The logs show a list of 17 tests, all of which passed. The tests are grouped into two categories: 'Schema' and 'Ini'. The 'Schema' category includes tests for missing names, unclosed braces, unexpected colons, and unexpected equals signs. The 'Ini' category includes tests for parsing empty files, single sections, single values, multiple sections, comments, and whitespace. The 'Ini' category also includes tests for missing values, missing sections, escape newlines, escape backslashes, malformed lines, and serialization. The final summary shows that 175 tests passed and 0 failed, with 433 assertions passed and 0 failed.

```
test
succeeded last week in 35s

Run tests 0s
167 [PASS] SchemaMissingNameFails
168 [PASS] SchemaUnclosedBraceFails
169 [PASS] SchemaUnexpectedColonFails
170 [PASS] SchemaUnexpectedEqualsFails
171 [PASS] IniParseEmpty
172 [PASS] IniParseSingleSection
173 [PASS] IniParseSingleValue
174 [PASS] IniParseMultipleSections
175 [PASS] IniParseComments
176 [PASS] IniTrimWhitespace
177 [PASS] IniMissingValue
178 [PASS] IniMissingSection
179 [PASS] IniEscapeNewline
180 [PASS] IniEscapeBackslash
181 [PASS] IniMalformedLine
182 [PASS] IniSerializeEmpty
183 [PASS] IniSerializeSingleSection
184 [PASS] IniSerializeMultipleValues
185 [PASS] IniSerializeMultipleSections
186 [PASS] IniSerializeEscapeNewline
187 [PASS] IniSerializeEscapeBackslash
188 [PASS] IniSerializeEscapeCarriageReturn
189 [PASS] IniRealWorld
190
191 175 passed, 0 failed
192 433 assertions passed, 0 failed
```

Exemplu de CI pentru un commit (231969e08ca67c8ee161fe482099d56bca584dea). Toate testele au trecut

Resurse Externe

Proiectul folosește exclusiv biblioteci open-source, incluse ca submodule Git în directorul vendor/:

Bibliotecă	Licență
Dear ImGui	MIT
GLFW	zlib/libpng
ImGuiColorTextEdit	MIT
ImGuiFileDialog	MIT

Sistemul de build: CMake.

Compilatoare/interpretoare externe folosite doar pentru a compila sau rula codul generat de utilizator (nu sunt distribuite împreună cu PVB): GCC, Clang sau MSVC pentru C++ și Python3 pentru Python.

În procesul de dezvoltare au fost utilizate instrumente bazate pe inteligență artificială exclusiv pentru activități de asistență tehnică, precum debugging, identificarea și analiza problemelor (de exemplu, memory leaks, erori de gestionare a memoriei, comportamente neașteptate și sugestii de diagnostic), precum și pentru verificarea unor soluții de implementare. Codul final, integrarea și validarea funcționalităților au fost realizate de autor.

Codul sursă PVB este licențiat sub GNU General Public License v3 și este disponibil public pe GitHub, la adresa github.com/mcostn/pvb.



Concluzii și opinia autorului

Ideea care a stat la baza proiectului PVB a pornit dintr-o observație simplă: pentru mulți elevi, cea mai dificilă etapă în învățarea programării nu este logica algoritmilor, ci trecerea de la reprezentarea vizuală a acestora la sintaxa unui limbaj de programare real. Platformele existente bazate pe blocuri facilitează înțelegerea conceptelor fundamentale, însă de cele mai multe ori nu explică în mod direct cum se transformă acestea în cod sursă compilabil. Din această nevoie a apărut PVB: o aplicație care nu încearcă să înlocuiască limbajele de programare clasice, ci să creeze o punte între gândirea vizuală și programarea textuală.

Consider că principalul avantaj al proiectului este faptul că utilizatorul poate observa, în timp real, corespondența dintre fiecare bloc și instrucțiunea de cod pe care aceasta o generează. În acest mod, procesul de învățare devine gradual: utilizatorul nu este obligat să memoreze din prima regulile de sintaxă, ci le descoperă în timp ce construiește algoritmi. Astfel, aplicația urmărește să reducă diferența dintre mediile educaționale bazate pe blocuri și dezvoltarea de aplicații folosind limbaje consacrate precum C++ sau Python.

Un exemplu concret de utilizare este introducerea structurilor de control într-o clasă de informatică. Un profesor poate construi împreună cu elevii un algoritm folosind blocuri vizuale, iar aceștia pot vedea imediat codul C++ sau Python generat pentru instrucțiuni precum **if**, **while** sau **for**. În acest fel, elevii înțeleg simultan atât logica algoritmului, cât și forma textuală pe care o vor întâlni ulterior la examene, concursuri sau în dezvoltarea de aplicații.

Consider că PVB demonstrează că programarea vizuală și programarea textuală nu trebuie privite ca două abordări separate, ci ca etape complementare ale aceluiași proces de învățare. Dacă aplicația va continua să fie dezvoltată și extinsă, aceasta poate deveni un instrument util atât pentru profesori, cât și pentru elevii aflați la începutul studiului informaticii, facilitând o tranziție mai naturală către programarea profesionistă.